

Apache Flink Runtime

核心流程与设计说明

—— 基于 Flink 1.15.4 源码深度分析 ——

模块	flink-runtime
版本	release-1.15.4
内容	核心流程图 + 关键类设计说明
范围	调度器/Checkpoint/资源管理/网络/状态

目录

第一章 Flink Runtime 整体架构概览

- 1.1 核心组件关系
- 1.2 集群启动流程

第二章 作业提交与调度

- 2.1 作业提交流程
- 2.2 调度器设计与调度策略
- 2.3 任务部署流程

第三章 Checkpoint 检查点机制

- 3.1 Checkpoint 触发与执行流程
- 3.2 Barrier 对齐机制
- 3.3 完成与失败处理

第四章 资源管理

- 4.1 声明式资源管理流程
- 4.2 Slot 分配与回收

第五章 Task 执行与故障恢复

- 5.1 Task 生命周期
- 5.2 故障恢复策略

第六章 核心类设计说明

- 6.1 ExecutionGraph 层次结构
- 6.2 Shuffle 与网络栈
- 6.3 State Backend 与内存管理
- 6.4 心跳机制

第一章 Flink Runtime 整体架构概览

1.1 核心组件关系

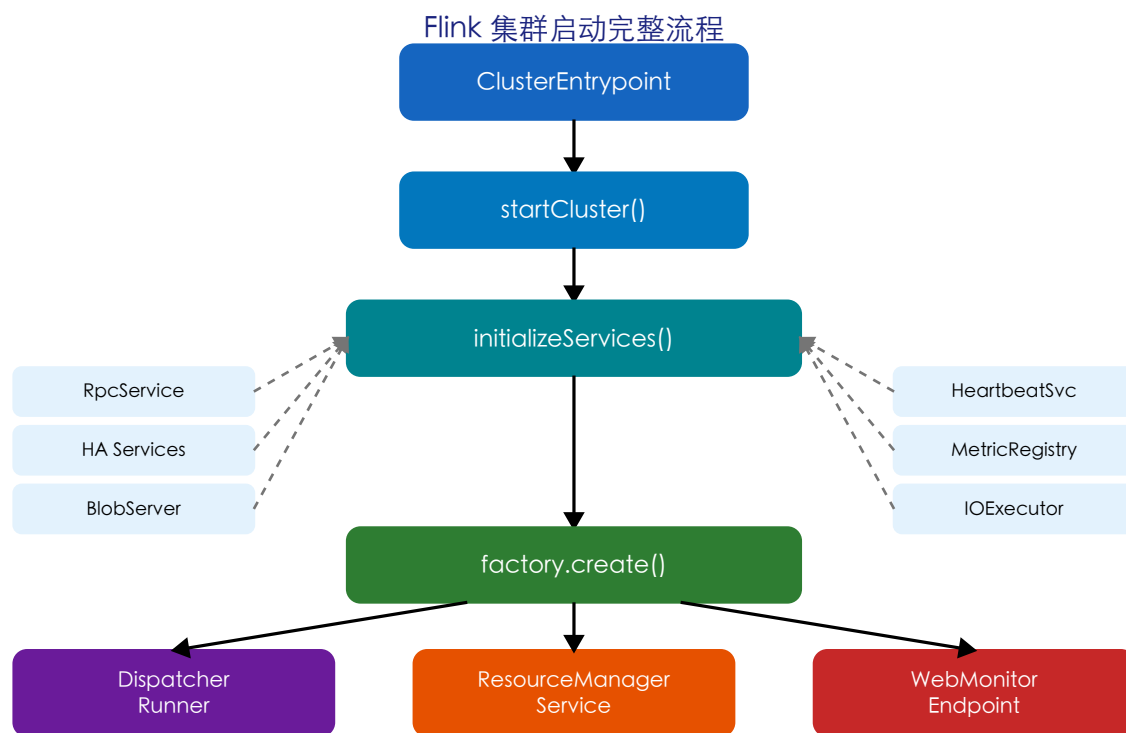
Flink Runtime 是 Apache Flink 的运行时核心，负责分布式作业的调度、执行、资源管理和容错。整个运行时由以下核心组件协作完成：

组件	所在进程	核心职责
ClusterEntrypoint	JobManager	集群启动入口，初始化所有基础服务
Dispatcher	JobManager	接收作业提交，管理 JobManagerRunner 生命周期
ResourceManager	JobManager	管理集群资源(Slot)，与外部框架(YARN/K8s)交互
JobMaster	JobManager	单个作业的调度、执行和 Checkpoint 协调
DefaultScheduler	JobManager	作业调度策略、Task 部署和故障恢复决策
TaskExecutor	TaskManager	管理 Slot、接收和执行 Task
Task	TaskManager	独立线程运行用户代码（如 StreamTask）

设计要点：Flink 采用 Master-Worker 架构。JobManager 进程包含 Dispatcher+ResourceManager+多个 JobMaster（每作业一个）；TaskManager 进程包含 TaskExecutor+多个 Task 线程。组件间通过 Akka/Pekko RPC 通信，核心方法在 ComponentMainThreadExecutor 中串行执行，避免并发问题。

1.2 集群启动流程

Flink 集群的启动从 ClusterEntrypoint.main() 开始，经过服务初始化后创建三大核心组件：

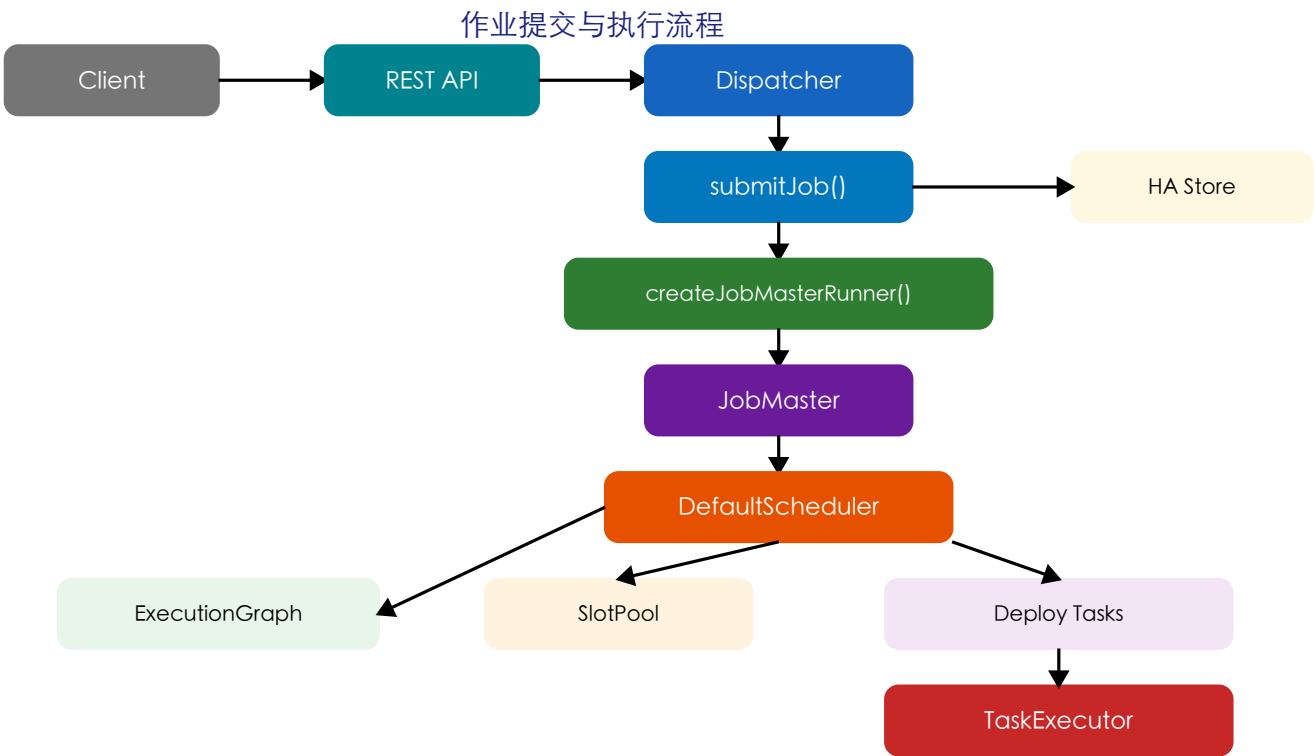


- `ClusterEntrypoint.runCluster()` — 顶层入口，配置安全上下文后进入核心流程
- `initializeServices()` — 创建 `RpcService`、`HA Services`、`BlobServer`、`HeartbeatServices` 等基础设施
- `DefaultDispatcherResourceManagerComponentFactory.create()` — 创建三大核心组件
- `DispatcherRunner` 通过 Leader 选举获取 Leadership 后创建 `Dispatcher` 实例
- `ResourceManagerService` 启动后参与 Leader 选举，管理集群资源

第二章 作业提交与调度

2.1 作业提交流程

客户端通过 REST API 将 JobGraph 提交给 Dispatcher，Dispatcher 持久化后创建 JobMaster 进行调度执行：



关键设计：作业提交时首先持久化到 HA Store（如 ZooKeeper），确保 JM 故障后可恢复未完成的作业。Dispatcher 为每个作业创建独立的 JobMasterServiceLeadershipRunner，通过 Leader 选举保证同一时刻只有一个 JobMaster 运行。

2.2 调度器设计与调度策略

DefaultScheduler 是 Flink 核心调度器，类层次：SchedulerNG(接口) → SchedulerBase(基类) → DefaultScheduler(实现)。

特性	PipelinedRegionStrategy	VertexwiseStrategy
调度粒度	Pipelined Region（一组顶点）	单个顶点
适用场景	流式作业（默认）	批处理作业
动态拓扑	不支持	支持（SchedulingTopologyListener）

特性	PipelinedRegionStrategy	VertexwiseStrategy
核心思想	同一Region内Task必须同时调度	逐顶点调度

2.3 任务部署流程

DefaultExecutionDeployer 负责将调度决策转化为实际部署，分为四步：

- Step 1: validateExecutionStates() — 验证所有 Execution 处于 CREATED 状态
- Step 2: transitionToScheduled() — 状态转为 SCHEDULED
- Step 3: allocateSlotsFor() — 异步分配 Slot (CompletableFuture<LogicalSlot>)
- Step 4: waitForAllSlotsAndDeploy() — Slot就绪后: assignResource→registerPartitions→deploy

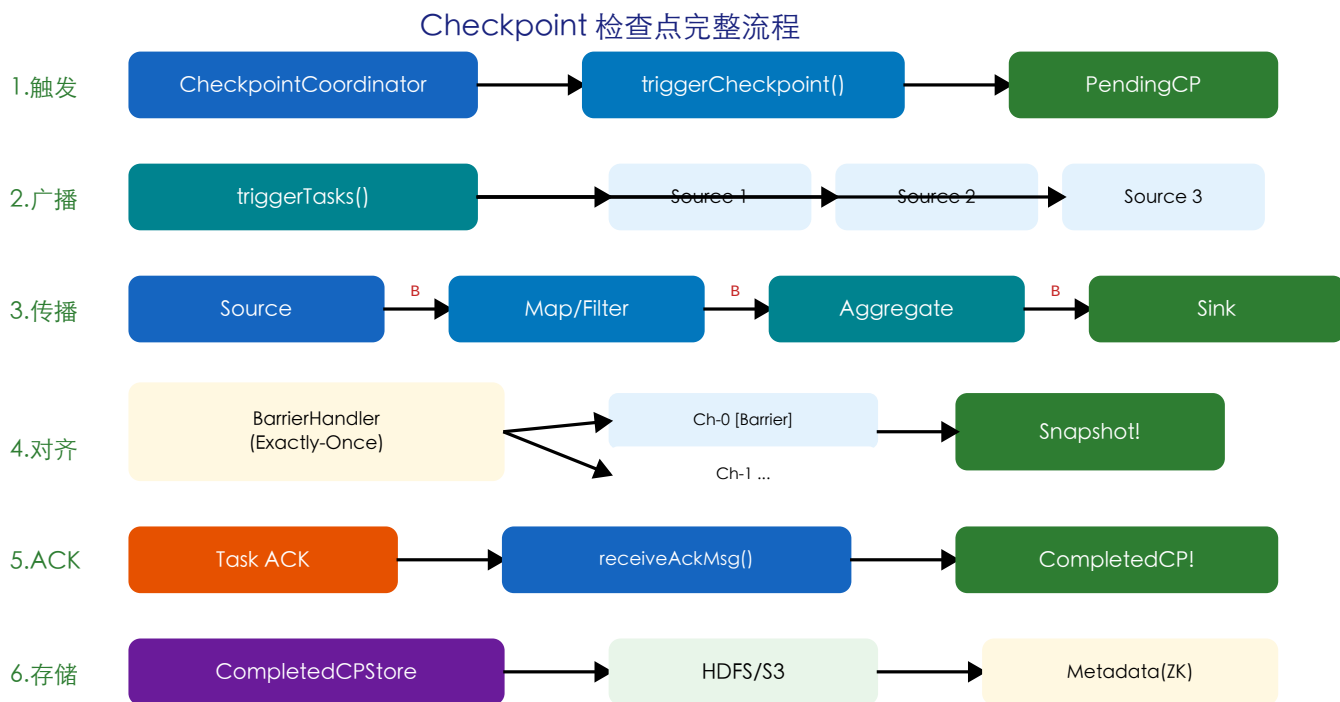
版本控制：ExecutionVertexVersioner

为每次调度/取消/故障恢复递增版本号，部署时检查版本是否过期，防止已取消的 Task 仍被部署。

第三章 Checkpoint 检查点机制

3.1 Checkpoint 触发与执行流程

Flink 的 Checkpoint 基于 Chandy-Lamport 分布式快照算法变体。CheckpointCoordinator 是 JM 端核心协调者：



3.2 Barrier 对齐机制

模式	实现类	行为	语义
Exactly-Once(对齐)	SingleCheckpointBarrierHandler	阻塞已收到Barrier通道，等待收齐	精确一次
At-Least-Once	CheckpointBarrierTracker	不阻塞通道，简单计数	至少一次
Unaligned(非对齐)	SingleCheckpointBarrierHandler	不阻塞，将未处理数据一并快照	精确一次(低延迟)

对齐超时自动降级：AlternatingCollectingBarriers

状态机支持对齐超时后自动切换非对齐模式，避免数据倾斜导致 Checkpoint 超时。

3.3 完成与失败处理

所有 Task ACK 收齐后，PendingCheckpoint 升级为 CompletedCheckpoint。CheckpointFailureManager 管理失败策略：

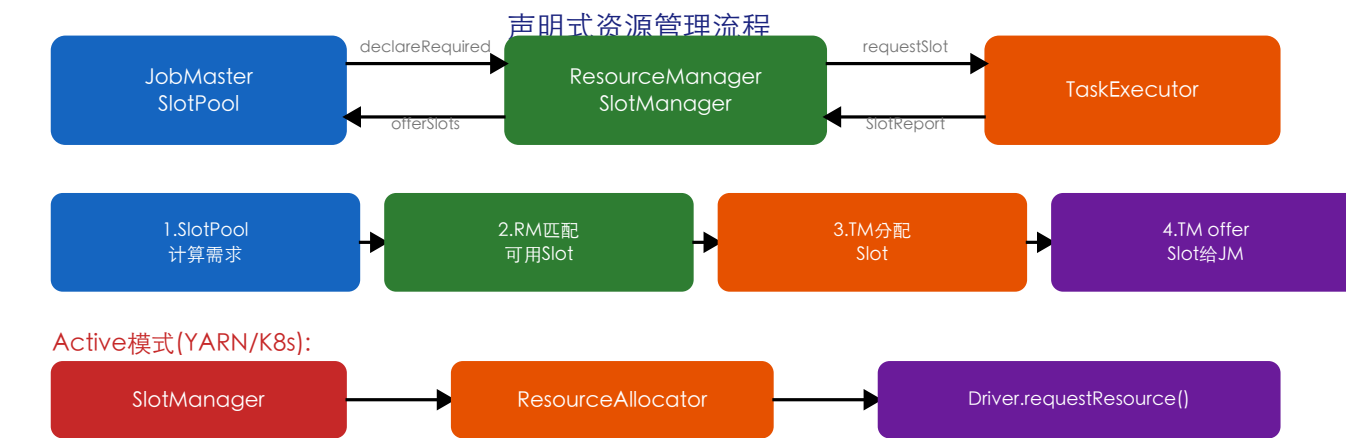
失败来源	CP类型	处理策略
JM级别	普通Checkpoint	计入失败计数，超阈值 failJob
JM级别	Savepoint	不做处理（手动操作）
TM级别	普通Checkpoint	计入失败计数
TM级别	同步Savepoint	直接 failJob（防死锁）

Savepoint vs Checkpoint：共享底层机制但有关键差异——Savepoint 由用户手动触发/管理，始终全量快照，不可被 subsume，不添加到 CompletedCheckpointStore。

第四章 资源管理

4.1 声明式资源管理流程

Flink 采用声明式资源管理模型。SlotPool 声明需求，SlotManager 匹配分配，ActiveRM 管理 Worker 生命周期：



方法	调用者	职责
registerJobMaster()	JobMaster	JM注册：验证Leader，建立心跳
registerTaskExecutor()	TaskExecutor	TM注册：创建WorkerRegistration
sendSlotReport()	TaskExecutor	接收Slot状态报告
declareRequiredResources()	JobMaster	接收资源需求声明

4.2 Slot 分配与回收

FineGrainedSlotManager

ResourceTracker(追踪Job需求)

SlotStatusSyncer(状态同步)。

的核心组件：TaskManagerTracker(追踪TM资源)

ResourceAllocationStrategy(匹配策略)

+

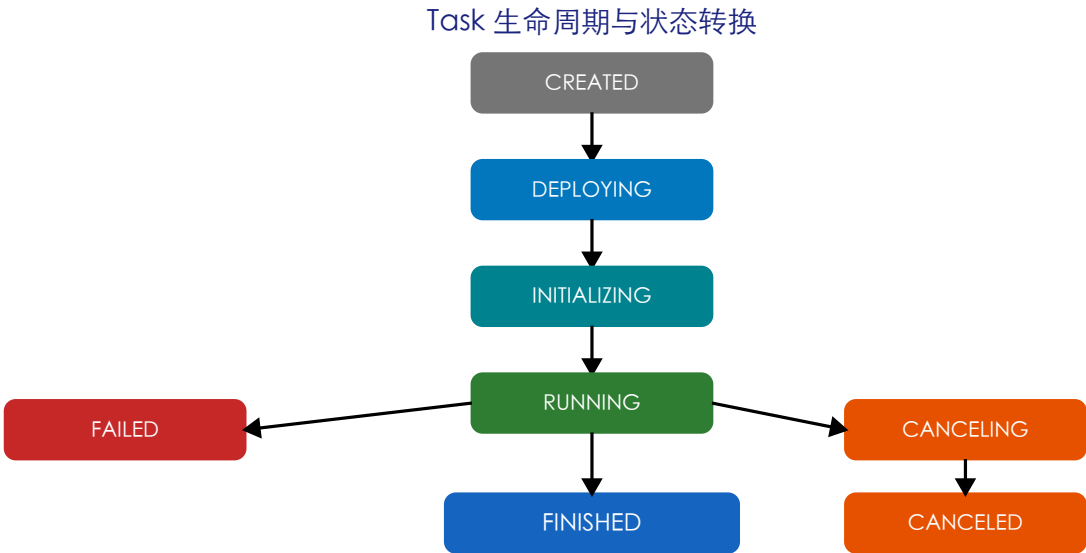
+

两种模式：Standalone 被动等待TM注册；Active(YARN/K8s) 通过 ResourceManagerDriver 主动申请 Worker，支持超时检查和冷却机制。

第五章 Task 执行与故障恢复

5.1 Task 生命周期与状态转换

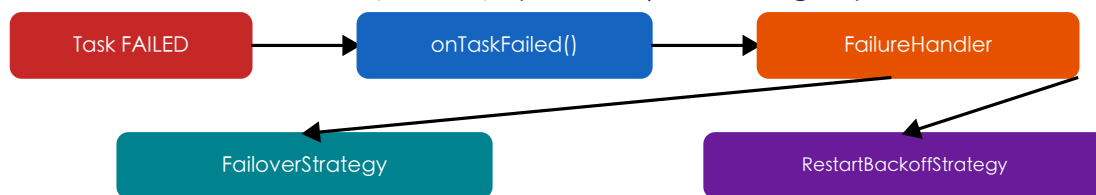
Task 是运行在 TaskManager 上的独立线程，通过 CAS 无锁机制保证状态转换原子性：



阶段	操作	说明
CREATED→DEPLOYING	创建类加载器	下载JAR，注册网络资源
DEPLOYING→INIT	创建RuntimeEnv	反射创建TaskInvokable(如StreamTask)
restore()	恢复状态	从Checkpoint恢复KeyedState和OperatorState
INIT→RUNNING	invoke()	执行用户代码主循环
RUNNING→终态	完成/失败/取消	释放资源，通知TaskManager最终状态

5.2 故障恢复策略

故障恢复流程 (RestartPipelinedRegion)



Region分析(BFS):



恢复执行:

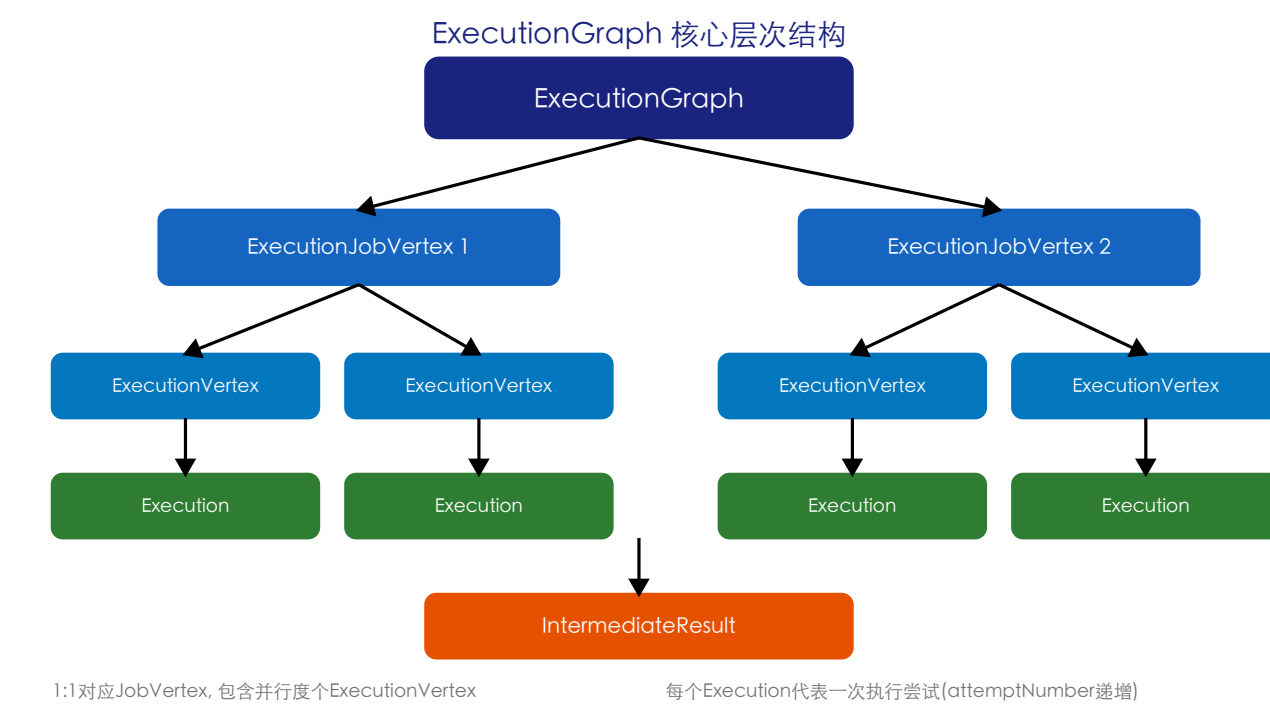


- 规则1：失败Task所在Region必须重启
- 规则2：如果Region的输入分区不可用，生产该分区的Region也需重启
- 规则3：如果一个Region需重启，其所有下游消费者Region也必须重启

状态恢复：全局恢复调用 `restoreLatestCheckpointedStateToAll()`；局部恢复调用 `restoreLatestCheckpointedStateToSubtasks()`，仅恢复受影响的 Task。

第六章 核心类设计说明

6.1 ExecutionGraph 层次结构



类名	对应关系	核心职责
DefaultExecutionGraph	整个作业	维护顶点、中间结果、作业状态、CheckpointCoordinator
ExecutionJobVertex	1:1 对应 JobVertex	一个算子逻辑表示，含并行度个 ExecutionVertex
ExecutionVertex	一个并行子任务	管理当前 Execution、历史记录和输出分区
Execution	一次执行尝试	状态转换、部署、取消，绑定 LogicalSlot
IntermediateResult	算子间数据集	上下游算子间传递的中间数据

两层状态管理：JM 侧 (Execution) 使用主线程串行更新，天然单线程保证；TM 侧 (Task) 使用 AtomicReferenceFieldUpdater CAS 无锁更新，允许执行线程和取消线程并发修改。

6.2 Shuffle 与网络栈

组件	所在端	核心职责
ShuffleMaster	JobMaster	分区注册、生成 ShuffleDescriptor (连接信息)
ShuffleEnvironment	TaskManager	创建 ResultPartitionWriter 和 InputGate
NetworkBufferPool	TaskManager	全局共享内存缓冲池 (堆外，默认 32KB/页)
ResultPartition	TaskManager	数据输出 (Pipelined/Blocking/SortMerge/Hybrid)
SingleInputGate	TaskManager	数据输入 (Local/Remote/UnknownChannel)

组件	所在端	核心职责
ConnectionManager	TaskManager	基于Netty的网络连接管理

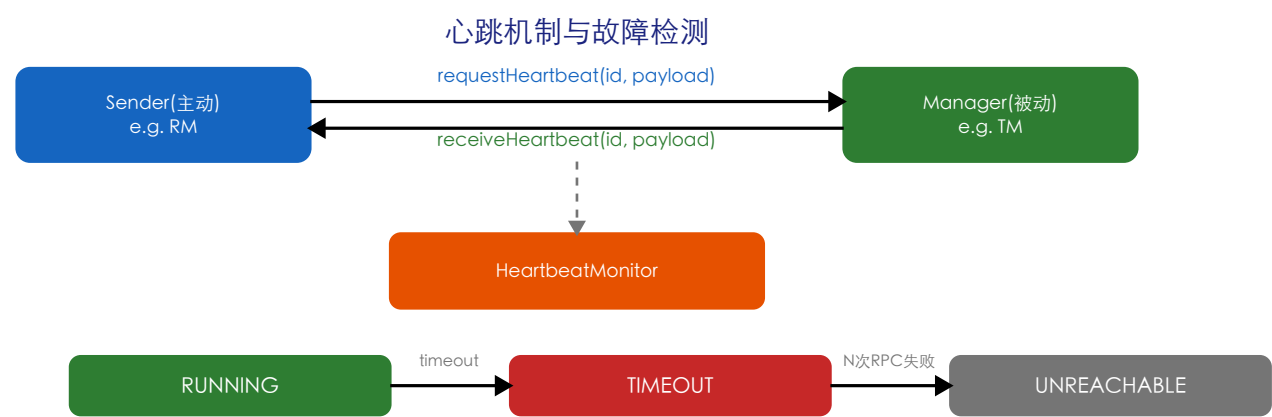
Credit-based流控：下游通过notifyCreditAvailable()告知上游可用缓冲区数量，上游仅在有信用时发送数据，实现端到端反压传播。

6.3 State Backend 与内存管理

分类	实现	存储位置	适用场景
KeyedStateBackend	HeapKeyedStateBackend	JVM堆内存	状态较小，低延迟
KeyedStateBackend	RocksDBKeyedStateBackend	本地RocksDB	TB级大状态，支持增量CP
CheckpointStorage	JobManagerCPStorage	JM内存	仅测试
CheckpointStorage	FileSystemCPStorage	HDFS/S3	生产推荐

- 预算控制：UnsafeMemoryBudget用CAS原子操作管理内存预算，防OOM
- 两种分配：MemorySegment(固定32KB页)和Chunk预留(RocksDB)
- 共享资源：SharedResources支持多算子共享内存（引用计数管理）
- Owner机制：内存绑定Owner，支持一次性释放和泄漏检测

6.4 心跳机制



心跳关系	主动方(Sender)	被动方	用途
RM↔TM	ResourceManager	TaskManager	资源可用性监控，携带SlotReport
JM↔TM	JobMaster	TaskManager	Task状态监控，携带累加器快照
RM↔JM	ResourceManager	JobMaster	作业健康监控

双重故障检测：DefaultHeartbeatMonitor同时支持时间超时(heartbeat.timeout=50s)和连续RPC失败两种故障检测方式，超时→TIMEOUT状态，N次RPC失败→UNREACHABLE状态。